```
$ iex
iex>
case "fail" do
  false -> nil
end

** (CaseClauseError) no case clause matching: "fail"
```

If you want to include a fallback in your case statement, capture the result into a variable:

```
$ iex
iex>
case "passed" do
  false -> nil
  var -> "It #{var}!"
end

"It passed!"
```

Use _ instead of a variable name if you want to match any value but you don't use the value in your code. This special variable name tells Elixir that you are intentionally not using the variable, so the compiler won't emit a warning.

Remember, you can have any number of case clauses in a case statement. It's a best practice to keep the number of clauses to a manageable size (up to three), but you have the ability to use as many as needed.

Case statements are used to match a single term against many patterns. There's a different control flow structure that adds pattern matching to a series of operations. Let's take a look at with statements next.

With Statements

A common task in programming is to run many functions and expect a successful result at each step. The final result of all of the functions together is what you want, but an error at any step results in the whole operation failing.

Elixir's with statements capture the successful outcome of a series of operations, plus there's a built-in way to handle any errors. Use with statements when you have several operations—especially if they are prone to failing—that must all succeed to return a result.

Here's an example of hypothetical code—that doesn't use a with statement—showing a chain of function calls:

```
{:ok, response} = MyApi.request()
{:ok, json} = Jason.decode(response)
{:ok, status} = Map.fetch!(json, "status")
```